SOLID AFRIQUE · SERVEUR MCP

Documentation technique

Architecture, sécurité, configuration, déploiement sur IIS et exploitation du serveur MCP d'accès aux bases de données.

Public	Équipe informatique : développeurs, administrateurs système et bases de données
Version	1.0 — octobre 2026
Code source	github.com/solid-afrique/serveurMCP_SOLID (branche main)
Pile	Laravel 13 · PHP 8.3 · MySQL · IIS (FastCGI)

Sommaire

1. Objet et périmètre
2. Architecture
3. Serveur MCP et outils
4. Authentification et autorisation
5. Protection des données
6. Modèle de données
7. Configuration
8. Déploiement sur IIS
9. Exploitation et dépannage
10. Développement et tests
11. Annexe : routes HTTP

> Le guide pas à pas de l'installation se trouve dans le dépôt : `deploy/DEPLOIEMENT-IIS.md` (inclus dans chaque paquet de déploiement). Le présent document en reprend l'essentiel et le complète par la description du fonctionnement interne.

1. Objet et périmètre

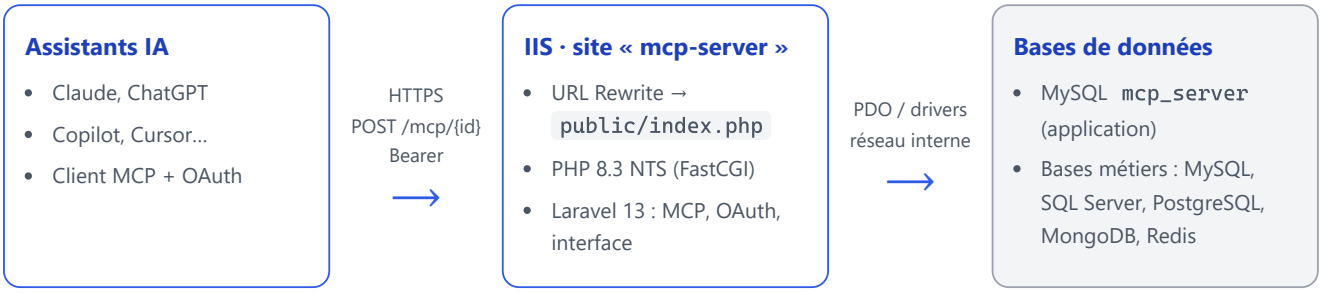
Le Serveur MCP expose des bases de données de l'entreprise à des assistants d'intelligence artificielle via le **Model Context Protocol** (MCP), standard ouvert qui permet à un assistant d'appeler des « outils » distants. Il comprend :

- un **serveur MCP** distant (transport *Streamable HTTP*), avec **une URL par connexion** de base de données ;
- un **serveur d'autorisation OAuth 2.1** : chaque assistant agit avec le compte de son utilisateur ;
- une **interface web d'administration** : connexions, utilisateurs, rôles, attribution des accès.

Élément	Prise en charge
Bases relationnelles	MySQL 5.7+ / MariaDB, Microsoft SQL Server (2012+, Azure SQL), PostgreSQL
Bases non relationnelles	MongoDB (si l'extension PHP `mongodb` est installée), Redis
Clients MCP testés	Microsoft 365 Copilot et Teams (via un agent Copilot Studio), Claude (claude.ai, Desktop, Claude Code), ChatGPT (mode développeur), Cursor ; tout client MCP gérant OAuth (ou via `mcp-remote`)
Hébergement	Windows Server + IIS (PHP en FastCGI), base MySQL pour les données de l'application
Historique	Une première version Next.js est archivée (branche `archive/nextjs`, étiquette `nextjs-v1`). La version actuelle est une réécriture complète en Laravel.

2. Architecture

2.1 Vue d'ensemble



Les bases métiers ne sont **pas exposées** sur Internet : seul le site IIS doit être joignable depuis l'extérieur, car Claude et ChatGPT appellent le serveur MCP depuis leurs propres infrastructures. Le serveur MCP joint ensuite les bases par le réseau interne.

2.2 Pile logicielle

Composant	Version	Rôle
PHP	8.3 ou 8.4 (NTS x64)	Exécution ; extensions requises : `pdo_mysql`, `sodium`, `openssl`, `mbstring`, `fileinfo`, `curl`
Laravel	13	Framework applicatif (routes, Eloquent, chiffrement, sessions, e-mails)
`laravel/mcp`	1.x	Serveur MCP (transport Streamable HTTP, outils, découverte OAuth)
`laravel/passport`	13 (league/oauth2-server 9)	Serveur OAuth 2.1 : autorisation, PKCE, jetons, enregistrement dynamique
`predis/predis`	3.x	Client Redis en PHP pur
`pdo_sqlsrv` + ODBC Driver 18	Microsoft	Accès SQL Server (facultatif)
`pdo_pgsql`, `mongodb`	PECL / PHP	PostgreSQL et MongoDB (facultatifs)
Interface	Blade + CSS/JS statiques	Pas de Node.js ni de compilation en production

2.3 Organisation du code

Emplacement	Contenu
`routes/ai.php`	Routes MCP (`Mcp::web('/mcp/{connection}')`) et routes OAuth (`Mcp::oauthRoutes()`)
`routes/web.php`	Interface : connexion, invitations, connexions, utilisateurs, compte
`app/Mcp/Servers/DatabaseServer.php`	Serveur MCP : nom et instructions adaptés à la connexion

`app/Mcp/Tools/`	Outils (`Sql/` , `Mongo/` , `Redis/` , `ConnectionInfo`) ; classe de base `DatabaseTool`
`app/Http/Middleware/AuthorizeMcpConnection.php`	Contrôle d'accès à chaque requête MCP
`app/Services/Databases/`	Accès aux bases : `SqlDatabase` (PDO), `MongoDatabase` , `RedisDatabase` , `ConnectionParameters` , `SqlGuard`
`app/Models/`	`User` , `DatabaseConnection` , `Invitation`
`app/Http/Controllers/`	Interface web (connexions, utilisateurs, compte, invitations, authentification)
`app/Mail/InvitationMail.php`	E-mail d'invitation / réinitialisation
`app/Console/Commands/`	`mcp:admin` , `mcp:mail-test`
`resources/views/`	Pages Blade, dont `mcp/authorize` (consentement OAuth)
`deploy/`	`package.ps1` , `DEPLOIEMENT-IIS.md` , scripts SQL
`public/web.config`	Configuration IIS (réécriture, erreurs, WebDAV, limites)

3. Serveur MCP et outils

3.1 Point d'accès

- **URL** : `https://luxewtey.com/mcpsolid/mcp/{connection}`, où `{connection}` est l'identifiant aléatoire (16 caractères) de la connexion.
- **Transport** : Streamable HTTP **sans état** ; chaque message JSON-RPC est une requête `POST`. `GET` et `DELETE` renvoient `405` (pas de flux SSE serveur ni de session).
- **Chaîne de traitement** : `auth:api` (jeton Passport) → `AuthorizeMcpConnection` → `DatabaseServer` → outil.

3.2 Traitement d'une requête

auth:api — vérifie le jeton d'accès (signature RSA, expiration, révocation). Sans jeton valide : `401` avec l'en-tête `WWW-Authenticate` et `resource_metadata`, qui déclenche la découverte OAuth côté client.

AuthorizeMcpConnection — charge la connexion (`404` si inconnue), vérifie que le titulaire du jeton est actif et y a accès (`403` sinon), que la connexion n'a pas expiré (`403`), puis met à jour `last_used_at` (au plus une fois par minute).

DatabaseServer::boot — fixe le nom du serveur et des instructions destinées au modèle : type de base, mode lecture seule, limite de lignes, consigne d'explorer la structure avant d'écrire des requêtes.

Outil — n'est exposé que s'il correspond au type de base et au mode (`DatabaseTool::shouldRegister`) ; ouvre la base, exécute, sérialise en JSON. Les annotations MCP (`readOnlyHint` , `destructiveHint`) sont renseignées.

3.3 Outils exposés

Base	Lecture	Écriture (connexion non lecture seule)
Toutes	`connection_info` : type, nom, mode, version	—
MySQL, SQL Server, PostgreSQL	`list_tables` , `describe_table` (colonnes, clés primaires / étrangères / uniques), `run_query` (paramètres `?`)	`execute_statement` (INSERT, UPDATE, DELETE, DDL ; confirmation demandée à l'utilisateur)
MongoDB	`list_databases` , `list_collections` , `describe_collection` (schéma déduit d'un échantillon, index), `find` , `count_documents` , `aggregate`	`insert_documents` , `update_documents` , `delete_documents` ; `$out` / `$merge` dans `aggregate`
Redis	`scan_keys` , `get_key` (tous types, TTL), `server_info` , `run_command` (liste blanche de commandes de lecture)	`run_command` (toutes commandes sauf liste noire)

3.4 Limites et formats

Paramètre	Valeur
Lignes / documents par appel	`max_rows` de la connexion : 200 par défaut, de 1 à 5 000

Taille de la réponse	100 000 caractères ; au-delà, le résultat est tronqué avec un message invitant à affiner la requête
Délai de connexion	10 s
Durée d'une requête SQL	30 s (`MAX_EXECUTION_TIME` MySQL, `max_statement_time` MariaDB, `statement_timeout` PostgreSQL, `QUERY_TIMEOUT` SQL Server)
Erreurs	Renvoyées à l'assistant sous forme de résultat d'outil en erreur (`Erreur : …`) ; aucune trace technique n'est exposée

4. Authentification et autorisation

4.1 Parcours de connexion d'un utilisateur

Un utilisateur s'authentifie deux fois, avec le **même compte** : sur l'interface web (session), puis, pour chaque assistant, par une autorisation OAuth. Le parcours complet :

#	Étape	Mécanisme
1	**Invitation** (administrateur)	`POST /users` crée le compte (`status = invited` , sans mot de passe) et envoie par e-mail le lien `https://luxewtey.com/mcpsolid/invitation/{jeton}` (usage unique, 7 jours). L'administrateur attribue les connexions (`PUT /connections/{id}/users`).
2	**Création du mot de passe**	`GET/POST /invitation/{jeton}` : mot de passe (10 caractères minimum, bcrypt), `status = active` , jeton supprimé, session ouverte.
3	**Connexion à l'interface**	`https://luxewtey.com/mcpsolid/login` : e-mail + mot de passe, session Laravel ; tentatives limitées (voir ci-dessous).
4	**Lien MCP**	Page *Mes connexions* : pour chaque connexion attribuée, l'URL `https://luxewtey.com/mcpsolid/mcp/{id}` à copier.
5	**Branchement de l'assistant**	L'assistant appelle l'URL MCP, reçoit `401` , découvre le serveur OAuth, s'enregistre, puis ouvre `/oauth/authorize` : l'utilisateur se connecte (session de l'étape 3) et clique sur **Autoriser**. L'assistant reçoit un jeton lié à ce compte (détails en 4.3).

Pour Microsoft Copilot, l'étape 5 est réalisée une fois par l'administrateur dans Copilot Studio (agent publié, voir 4.5), puis chaque collaborateur autorise son propre compte à la première utilisation de l'agent.

4.2 Interface web

- Session Laravel (pilote `database`), cookie `Secure` en HTTPS, protection CSRF sur tous les formulaires.
- Mots de passe hachés en **bcrypt** (12 tours), 10 caractères minimum.
- **Limitation des tentatives** : 10 échecs par couple IP + e-mail, et 100 échecs au total, sur 15 minutes. Une empreinte factice est vérifiée pour les comptes inconnus, afin que le temps de réponse ne révèle pas l'existence d'un compte.
- **Invitations** : jeton aléatoire de 48 caractères, dont seule l'empreinte SHA-256 est stockée ; usage unique ; 7 jours ; un nouveau lien invalide le précédent. Envoi par e-mail SMTP (`InvitationMail`) ; en mode `MAIL_MAILER=log` , rien n'est envoyé (pour ne pas écrire le lien dans les journaux) et le lien est affiché à l'administrateur.
- Aucune inscription libre : les comptes sont créés par un administrateur, ou par la commande `php artisan mcp:admin` .

4.3 OAuth 2.1 pour les assistants

Élément	Mise en œuvre
Métadonnées de la ressource (RFC 9728)	`/.well-known/oauth-protected-resource/…`

Métadonnées du serveur (RFC 8414)	`/.well-known/oauth-authorization-server`
Enregistrement dynamique (RFC 7591)	`POST /oauth/register` : clients publics, sans secret
Autorisation	`/oauth/authorize` : code d'autorisation + **PKCE S256** obligatoire ; écran de consentement `resources/views/mcp/authorize.blade.php`
Jetons	`/oauth/token` ; jeton d'accès (JWT signé RS256) valable **1 heure** ; jeton de rafraîchissement **30 jours**
Portée	`mcp:use`
Adresses de retour autorisées	`MCP_REDIRECT_DOMAINS` (par défaut : claude.ai, claude.com, chatgpt.com, chat.openai.com, vscode.dev, insiders.vscode.dev, global.consent.azure-apim.net pour Copilot Studio, localhost, 127.0.0.1) et schémas personnalisés `claude` , `cursor` , `vscode` , `vscode-insiders`
Clés de signature	`storage/oauth-private.key` et `storage/oauth-public.key` (`php artisan passport:keys`)

Déroulement d'une première connexion

- **Assistant** — `POST /mcp/{id}` sans jeton → `401` + `WWW-Authenticate: Bearer resource_metadata=…`
- **Assistant** — lit les métadonnées (RFC 9728 puis RFC 8414), s'enregistre via `/oauth/register` .
- **Navigateur** — ouvre `/oauth/authorize` (avec `code_challenge` et `resource`) ; l'utilisateur se connecte si nécessaire, puis voit l'écran de consentement (nom du client, connexion visée, compte, domaine de retour).
- **Utilisateur** — clique sur **Autoriser** → redirection vers l'assistant avec un code.
- **Assistant** — échange le code + `code_verifier` contre les jetons, puis rejoue `POST /mcp/{id}` avec `Authorization: Bearer` .

4.4 Règles d'accès

Règle	Implémentation
Déclarer / gérer une connexion	`canBeManagedBy` : administrateurs
Utiliser une connexion (MCP)	`canBeUsedBy` : compte actif *et* (administrateur *ou* attributaire via `connection_user`)
Attribution	Une connexion déclarée par un administrateur n'est utilisable que par les administrateurs tant qu'elle n'est attribuée à aucun utilisateur (*Gérer l'accès*)
Révocation	Vérifiée à **chaque** requête MCP : retrait d'attribution, désactivation de compte ou retrait de rôle prennent effet immédiatement. La désactivation supprime aussi les sessions et révoque jetons d'accès et de rafraîchissement.

> Une autorisation OAuth est accordée **par utilisateur et par application** (et non par connexion) : un assistant autorisé peut utiliser toutes les connexions auxquelles l'utilisateur a accès, dans la limite des URL qui lui ont été configurées.

4.5 Intégration Microsoft Copilot

Microsoft 365 Copilot (application Copilot, Teams, Outlook) n'accepte pas d'URL MCP directement : le serveur est ajouté comme **outil MCP d'un agent Copilot Studio**, publié ensuite sur le canal Microsoft 365 Copilot et Teams.

Élément	Mise en œuvre
Outil de création	**Copilot Studio** (copilotstudio.microsoft.com) : *Outils → Ajouter un outil → Nouvel outil → Model Context Protocol*. L'« Agent Builder » intégré à Microsoft 365 Copilot ne gère pas le MCP.
Prérequis de l'agent	Orchestration générative activée ; licence ou essai Copilot Studio ; droit de créer des connecteurs personnalisés dans l'environnement Power Platform (aucune stratégie DLP ne doit les bloquer).
Authentification	OAuth 2.0 → **Découverte dynamique** : Copilot Studio lit les métadonnées RFC 9728 / 8414 et s'enregistre via `/oauth/register`, avec PKCE.
Adresse de retour	`https://global.consent.azure-apim.net/redirect/…` (propre à chaque connecteur). Le domaine figure dans la liste par défaut de `MCP_REDIRECT_DOMAINS` ; une liste personnalisée dans `.env` doit le reprendre.
Identité utilisée	Configurer l'outil avec l'**authentification de l'utilisateur final** : chaque collaborateur autorise son propre compte et les règles d'accès du serveur s'appliquent à lui. Avec les identifiants du créateur, tous les utilisateurs de l'agent partageraient les droits de ce dernier.
Diffusion	Un agent par base (ou par domaine fonctionnel), publié par un administrateur ; les utilisateurs l'ouvrent depuis la liste des agents de Copilot et autorisent leur compte à la première utilisation.

5. Protection des données

5.1 Paramètres des connexions

- Stockés dans `database_connections.config` avec le cast `encrypted:array` : chiffrement **AES-256-CBC** + HMAC avec `APP_KEY`.
- Les mots de passe et chaînes de connexion ne sont **jamais renvoyés** au navigateur (`publicConfig()` ne transmet que des indicateurs `has_password` / `has_connection_string`). En modification, un champ vide conserve le secret existant.
- **Perte d'`APP_KEY` = perte des paramètres** de toutes les connexions. La clé doit être sauvegardée hors du serveur et ne jamais être régénérée une fois des connexions créées.

5.2 Mode lecture seule

Une connexion en lecture seule n'expose pas les outils d'écriture, et chaque requête est en outre protégée à deux niveaux :

Niveau	Mécanisme
Transaction	MySQL : `START TRANSACTION READ ONLY` ; PostgreSQL : `BEGIN TRANSACTION READ ONLY` ; SQL Server : transaction systématiquement annulée (`ROLLBACK`).
Garde-fou SQL	`SqlGuard` : une seule instruction (hors littéraux et commentaires), qui doit commencer par `SELECT` , `WITH` , `SHOW` , `EXPLAIN` , `DESCRIBE` , `VALUES` ou `TABLE` ; refus des mots-clés d'écriture ou dangereux (`INSERT` , `UPDATE` , `INTO` , `EXEC` , `xp_*` , `sp_*` , `OPENROWSET` , `LOAD_FILE` , `SLEEP` , `WAITFOR` …).
Pilote	MySQL : `MULTI_STATEMENTS` désactivé, requêtes préparées natives.
MongoDB	Seuls les outils de lecture sont exposés ; `$out` et `$merge` sont refusés dans `aggregate` .
Redis	`run_command` limité à une liste blanche de commandes de lecture ; liste noire permanente (`SHUTDOWN` , `MONITOR` , `ACL` , `MODULE` , commandes bloquantes…).

> **Défense en profondeur.** Ces protections ne remplacent pas les droits de la base. Utilisez des **comptes de base en lecture seule** dédiés : scripts `deploy/sql/mysql-lecteur.sql` et `deploy/sql/sqlserver-lecteur.sql` .

5.3 Autres mesures

- `APP_DEBUG=false` en production : aucune trace d'erreur détaillée n'est renvoyée (les réponses MCP et OAuth sont toujours en JSON).
- HTTPS obligatoire (`URL::forceScheme('https')` lorsque `APP_URL` est en https) ; cookies de session sécurisés.
- `TRUSTED_PROXIES` à renseigner uniquement si un proxy inverse est placé devant IIS.
- **Flux de données** : les résultats des requêtes sont transmis au fournisseur de l'assistant (Anthropic, OpenAI, Microsoft). Le choix des connexions exposées et des colonnes accessibles aux comptes lecteurs doit en tenir compte (données personnelles, données sensibles).

6. Modèle de données

Base MySQL de l'application (`mcp_server` par défaut), créée par `php artisan migrate` .

Table	Colonnes principales
`users`	`id` , `name` , `email` (unique), `password` (nul tant que l'invitation n'est pas acceptée), `role` (`admin` | `user`), `status` (`invited` | `active` | `disabled`), `last_login_at`
`database_connections`	`id` (chaîne aléatoire de 16 caractères, utilisée dans l'URL MCP), `name` , `type` (`mysql` | `sqlsrv` | `pgsql` | `mongodb` | `redis`), `read_only` , `max_rows` , `config` (texte chiffré), `owner_id` (nul = partagée ; suppression en cascade avec le propriétaire), `expires_at` , `last_used_at`
`connection_user`	Attribution des connexions partagées aux utilisateurs (`database_connection_id` , `user_id`)
`invitations`	`user_id` (unique), `token_hash` (SHA-256), `expires_at`
`oauth_clients`	Clients enregistrés dynamiquement (nom, adresses de retour, types d'octroi)
`oauth_auth_codes` , `oauth_access_tokens` , `oauth_refresh_tokens`	Codes et jetons émis (Passport) ; la page « Mon compte » et la révocation s'appuient sur ces tables
`sessions` , `cache` , `cache_locks`	Sessions web et cache (limitation des tentatives)
`jobs` , `failed_jobs` …	Tables standard de Laravel (non utilisées : `QUEUE_CONNECTION=sync`)

7. Configuration

Fichier `.env` à la racine de l'application (modèle : `.env.example`). Après toute modification : `php artisan config:cache`, puis recyclage du pool IIS.

Variable	Rôle / valeur recommandée
`APP_ENV` / `APP_DEBUG`	`production` / `false`
`APP_KEY`	Générée par `php artisan key:generate`. Chiffre sessions et paramètres des connexions. **À sauvegarder.**
`APP_URL`	Adresse publique HTTPS, sans « / » final (sert à construire les URL MCP et OAuth)
`DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	Base MySQL de l'application. Mot de passe entre guillemets s'il contient des espaces ou `# $ " =`
`SESSION_SECURE_COOKIE`	`true` (HTTPS)
`LOG_CHANNEL`, `LOG_LEVEL`	`daily` (14 jours), `warning`
`MAIL_MAILER`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	SMTP pour les invitations (Microsoft 365 : `smtp.office365.com`:587). `MAIL_SCHEME=smtps` pour le port 465. `MAIL_TIMEOUT` : 15 s par défaut. `log` = pas d'envoi
`MCP_REDIRECT_DOMAINS`	Domaines de retour OAuth autorisés, séparés par des virgules (voir chapitre 4)
`TRUSTED_PROXIES`	Uniquement derrière un proxy inverse (ARR, répartiteur) : IP du proxy ou `*`
`QUEUE_CONNECTION`, `CACHE_STORE`, `SESSION_DRIVER`	`sync`, `database`, `database` (aucun service supplémentaire requis)

8. Déploiement sur IIS

Déploiement **manuel** par paquet .zip : ni Git, ni Composer, ni Node.js sur le serveur. Procédure détaillée : `deploy/DEPLOIEMENT-IIS.md`.

8.1 Prérequis du serveur

- IIS avec la fonctionnalité **CGI** et le module **URL Rewrite 2.1**.
- PHP 8.3/8.4 **NTS x64** + Visual C++ Redistributable ; `php.ini` avec les extensions du chapitre 2, `cgi.fix_pathinfo=0`, `fastcgi.impersonate=1`, OPcache activé.
- SQL Server : **ODBC Driver 18** + DLL `php_pdo_sqlsrv` correspondant exactement à la version de PHP et à la variante NTS.
- MySQL pour la base de l'application ; nom d'hôte public, DNS et certificat TLS ; site joignable depuis Internet.

8.2 Construction du paquet (poste de développement)

```
powershell -ExecutionPolicy Bypass -File deploy\package.ps1
```

Le script copie le projet sans les fichiers de développement (`.git`, `tests`, `docs`, `.env`, clés, journaux), exécute `composer install --no-dev --optimize-autoloader --classmap-authoritative` et produit `dist\mcp-server-<date>-<commit>.zip` (environ 10 Mo).

8.3 Première installation

1. Créer la base et le compte MySQL : `deploy\sql\mysql-application.sql` (remplacer `CHANGEZ-MOI`).
2. Débloquer puis extraire le .zip (ex. `E:\Sites\mcp-server`) ; copier `.env.example` en `.env` et le compléter.
3. Dans une invite **administrateur**, à la racine de l'application :

```
php artisan key:generate --force
php artisan migrate --force
php artisan passport:keys
php artisan mcp:admin admin@domaine.com --name="Administrateur"
php artisan mcp:mail-test admin@domaine.com
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

4. Pool d'applications `mcp-server` : **Aucun code managé**, mode intégré, démarrage `AlwaysRunning`, délai d'inactivité 0.
5. Droits : lecture/exécution sur le dossier, **modification** sur `storage` et `bootstrap\cache` pour `IIS AppPool\mcp-server` (et pour le compte qui lance les commandes Artisan).
6. Site IIS : chemin physique = dossier `public`, liaison HTTPS avec SNI ; authentification anonyme = identité du pool.
7. Vérifier depuis l'extérieur : `/up`, `/.well-known/oauth-authorization-server`, page de connexion.

8.4 Installation de production

Élément	Valeur
Adresse publique (`APP_URL`)	`https://luxewtey.com/mcpsolid`
IIS	Application `mcpsolid` sous le site `luxewtey.com` ; fichiers dans `E:\NOS SITES\MCPSOLID` (chemin physique : dossier `public`)
URL MCP d'une connexion	`https://luxewtey.com/mcpsolid/mcp/<identifiant>` (affichée dans l'interface)
Interface d'administration	`https://luxewtey.com/mcpsolid/login`
Sonde de disponibilité	`https://luxewtey.com/mcpsolid/up`
Découverte OAuth	`https://luxewtey.com/mcpsolid/.well-known/oauth-authorization-server` (émetteur `https://luxewtey.com/mcpsolid`) ; métadonnées de ressource annoncées dans l'en-tête `WWW-Authenticate` : `https://luxewtey.com/mcpsolid/.well-known/oauth-protected-resource/mcp/<identifiant>`

> **Installation sous un sous-chemin.** Certains clients appliquent la RFC 8414 à la lettre et cherchent les métadonnées à la racine du domaine : `https://luxewtey.com/.well-known/oauth-authorization-server/mcpsolid` . Le site racine `luxewtey.com` redirige actuellement cette adresse vers `https://luxewtey.com/mcpsolid/.well-known/oauth-authorization-server` : cette redirection doit être conservée lors des évolutions du site racine.

8.5 Mise à jour

```
php artisan down
robocopy C:\Temp\mcp-nouveau <dossier du site> /E /XF .env /XD storage
php artisan migrate --force
php artisan config:cache & php artisan route:cache & php artisan view:cache
php artisan up
```

`.env` , `storage\` (journaux, clés OAuth) et la base MySQL sont conservés.

9. Exploitation et dépannage

9.1 Tâches courantes

Besoin	Commande / emplacement
Journaux applicatifs	`storage\logs\laravel-AAAA-MM-JJ.log`
Créer ou réparer un administrateur	`php artisan mcp:admin email@domaine.com` (réactive le compte, le nomme administrateur, redéfinit le mot de passe)
Tester l'envoi d'e-mails	`php artisan mcp:mail-test email@domaine.com`
Purger les jetons expirés	`php artisan passport:purge` (tâche planifiée hebdomadaire conseillée)
Maintenance	`php artisan down` / `php artisan up`
Après modification de `.env`	`php artisan config:cache`, puis recycler le pool

9.2 Sauvegardes

- Base MySQL `mcp_server` (sauvegarde quotidienne).
- Fichier `.env`, en particulier `APP_KEY` — **indispensable** pour relire les paramètres des connexions.
- `storage\oauth-private.key` et `storage\oauth-public.key` (sans elles, les assistants doivent simplement être réautorisés).

9.3 Dépannage

Symptôme	Cause et solution
Erreur 500.19 (IIS)	Module URL Rewrite absent : l'installer.
404 IIS sur `/login`	Le site pointe sur la racine au lieu de `public`, ou la réécriture est inactive.
« No application encryption key has been specified »	`APP_KEY` vide ou configuration mise en cache avant la génération de la clé : `config:clear`, `key:generate --force` (uniquement s'il n'existe encore aucune connexion), `config:cache`, recyclage du pool. Vérifier que le fichier s'appelle bien `.env` (et non `.env.txt`).
« The stream or file … could not be opened »	Droits d'écriture manquants sur `storage` / `bootstrap\cache` : `icacls … /grant "IIS AppPool\mcp-server:(OI)(CI)M"`.
`SQLSTATE[HY000] [1045] Access denied`	Compte ou mot de passe MySQL de l'application incorrect ; guillemets manquants dans `.env` ; compte existant créé avec un autre mot de passe (`ALTER USER … IDENTIFIED BY …`).
« Unable to load dynamic library pdo_sqlsrv »	DLL absente ou ne correspondant pas à la version / variante de PHP. Corriger la DLL ou commenter la ligne si SQL Server n'est pas utilisé.
`could not find driver` au test d'une connexion	Extension PDO du type de base non chargée (`php -m`).

L'assistant n'arrive pas à se connecter	`APP_URL` différent de l'adresse publique, site non joignable depuis Internet, ou `/.well-known/…` bloqué. Vérifier depuis l'extérieur.
« redirect_uris … not a permitted redirect domain »	L'adresse refusée est journalisée dans `storage\logs` (« Enregistrement OAuth refusé »). Ajouter son domaine à `MCP_REDIRECT_DOMAINS` (en reprenant la liste par défaut), puis `config:cache`.
Copilot Studio : « GetDynamicClientRegistrationResultAsync failed … invalid_redirect_uri »	`MCP_REDIRECT_DOMAINS` personnalisé sans `https://global.consent.azure-apim.net` : l'ajouter (en reprenant la liste par défaut), `config:cache`, recycler le pool, puis recréer la connexion de l'outil.
Invitation : « L'e-mail n'a pas pu être envoyé »	`mcp:mail-test` affiche l'erreur SMTP exacte (identifiants, port 587 bloqué, SMTP authentifié désactivé sur la boîte Microsoft 365).

10. Développement et tests

10.1 Poste de développement

```
composer install
copy .env.example .env     (APP_ENV=local, APP_DEBUG=true, APP_URL=http://127.0.0.1:8000,
SESSION_SECURE_COOKIE=false, DB_*)
php artisan key:generate
php artisan migrate
php artisan passport:keys
php artisan mcp:admin vous@exemple.com
cd public
php -S 127.0.0.1:8000 ..\vendor\laravel\framework\src\Illuminate\Foundation\resources\server.php
```

Si l'extension `sodium` n'est pas activée (WAMP), préfixer les commandes par `php -d extension=sodium` ; désactiver Xdebug (`XDEBUG_MODE=off`).

10.2 Tests

- `php artisan test` : 33 tests PHPUnit (garde-fou SQL, analyse des chaînes de connexion, contrôles d'accès, invitations et e-mails, protection OAuth du point MCP), sur SQLite en mémoire.
- Validation de bout en bout réalisée avec le client OAuth du SDK MCP officiel (découverte, enregistrement, PKCE, appels d'outils, révocation) et sur les cinq moteurs de base (SQL Server, MySQL, PostgreSQL, MongoDB, Redis).

10.3 Ajouter un outil MCP

1. Créer une classe dans `app/Mcp/Tools/…` héritant de `DatabaseTool` ; renseigner `$name` , `$description` , `$types` (bases concernées) et `$writes` (outil d'écriture).
2. Déclarer le schéma d'entrée et implémenter `handle()` en passant par `$this->run(fn () => …)` (sérialisation, troncature, gestion d'erreurs).
3. L'ajouter à `DatabaseServer::$tools` et écrire un test.

11. Annexe : routes HTTP

Chemins relatifs à l'adresse de l'application, `https://luxewtey.com/mcpsolid` (par exemple `https://luxewtey.com/mcpsolid/oauth/token`).

Méthode	Chemin	Rôle
POST	`/mcp/{connection}`	Point MCP (jeton OAuth requis)
GET	`/.well-known/oauth-protected-resource/…`	Métadonnées de la ressource
GET	`/.well-known/oauth-authorization-server`	Métadonnées du serveur d'autorisation
POST	`/oauth/register`	Enregistrement dynamique des clients
GET / POST / DELETE	`/oauth/authorize`	Consentement (approuver / refuser)
POST	`/oauth/token`	Émission et rafraîchissement des jetons
GET / POST	`/login`, `/logout`	Session de l'interface
GET / POST	`/invitation/{token}`	Activation / réinitialisation du mot de passe
—	`/connections` (ressource), `/connections/test`, `/connections/{id}/users`	Connexions, test, attribution (admin)
—	`/account`, `/account/password`, `/account/assistants/{client}`	Compte, mot de passe, révocation d'un assistant
—	`/users`, `/users/{id}`, `/users/{id}/invitation`, `/users/{id}/assistants`	Gestion des comptes (admin)
GET	`/up`	Sonde de disponibilité